Quaternion Basics for Robotics

EE0849 Introduction to Robotics — Supplementary Notes

Basri Erdogan

Table of contents

1 What is a Quaternion? 1

2 Unit Quaternions and Rotations 3
- 2.1 Examples . . . 3
- 2.2 Key Observation . . . 3

3 Quaternion Operations 4
- 3.1 Conjugate . . . 4
- 3.2 Multiplication . . . 4
- 3.3 Rotating a Vector . . . 4
- 3.4 Conversion to Rotation Matrix . . . 4

4 Why Quaternions? 5
- 4.1 Comparison with Other Representations . . . 5
- 4.2 Advantages of Quaternions . . . 5
- 4.3 Where You Will See Them . . . 5

5 SLERP: Spherical Linear Interpolation 6
- 5.1 The Problem . . . 6
- 5.2 The Formula . . . 6
- 5.3 Properties . . . 6
- 5.4 Practical Note: Check the Dot Product . . . 6

6 Python Examples 7
- 6.1 Creating Quaternions . . . 7
- 6.2 Operations . . . 7
- 6.3 SLERP . . . 7

7 Summary 9

1 What is a Quaternion?

A **quaternion** is a 4-component number system that extends complex numbers:

$$\mathbf{q} = w + x\,\mathbf{i} + y\,\mathbf{j} + z\,\mathbf{k}$$

where **i**, **j**, **k** are imaginary units with the property:

$$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1$$

We often write a quaternion as a **scalar-vector pair**:

$$\mathbf{q} = (w,\ \mathbf{v}) \quad \text{where } \mathbf{v} = (x, y, z)$$

The **norm** (magnitude) is:

$$\|\mathbf{q}\| = \sqrt{w^2 + x^2 + y^2 + z^2}$$

2 Unit Quaternions and Rotations

A **unit quaternion** has $\|\mathbf{q}\| = 1$ and represents a 3D rotation.

Given a rotation by angle θ about a unit axis $\hat{\mathbf{k}} = (k_x, k_y, k_z)$:

$$\mathbf{q} = \cos\frac{\theta}{2} + \sin\frac{\theta}{2}\,(k_x\,\mathbf{i} + k_y\,\mathbf{j} + k_z\,\mathbf{k})$$

or equivalently:

$$\mathbf{q} = \left(\cos\frac{\theta}{2},\ \ \sin\frac{\theta}{2}\,\hat{\mathbf{k}}\right)$$

2.1 Examples

No rotation (identity):

$$\mathbf{q} = (1,\ 0,\ 0,\ 0)$$

90° about the z-axis:

$$\mathbf{q} = (\cos 45°,\ 0,\ 0,\ \sin 45°) = \left(\frac{\sqrt{2}}{2},\ 0,\ 0,\ \frac{\sqrt{2}}{2}\right)$$

180° about the x-axis:

$$\mathbf{q} = (\cos 90°,\ \sin 90°,\ 0,\ 0) = (0,\ 1,\ 0,\ 0)$$

2.2 Key Observation

The **half-angle** $\theta/2$ appears because quaternions double-cover SO(3): both $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation. This is not a problem in practice — just pick one convention.

3 Quaternion Operations

3.1 Conjugate

$$\mathbf{q}^* = (w,\ -\mathbf{v}) = w - x\,\mathbf{i} - y\,\mathbf{j} - z\,\mathbf{k}$$

For a unit quaternion, the conjugate is the **inverse rotation**: $\mathbf{q}^{-1} = \mathbf{q}^*$.

3.2 Multiplication

Quaternion multiplication composes rotations (like matrix multiplication, but with 4 components):

$$\mathbf{q}_1 \otimes \mathbf{q}_2 = (w_1 w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2,\ \ w_1\mathbf{v}_2 + w_2\mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2)$$

Important: Multiplication is **not commutative**: $\mathbf{q}_1 \otimes \mathbf{q}_2 \neq \mathbf{q}_2 \otimes \mathbf{q}_1$ (just like rotation matrices).

3.3 Rotating a Vector

To rotate a vector $\mathbf{p}$ by quaternion $\mathbf{q}$:

$$\mathbf{p}' = \mathbf{q} \otimes (0,\ \mathbf{p}) \otimes \mathbf{q}^*$$

The result's vector part gives the rotated vector.

3.4 Conversion to Rotation Matrix

A unit quaternion $\mathbf{q} = (w, x, y, z)$ converts to:

$$R = \begin{bmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{bmatrix}$$

4 Why Quaternions?

4.1 Comparison with Other Representations

	Rotation Matrix	Euler/RPY Angles	Quaternion
Parameters	9 (with 6 constraints)	3	4 (with 1 constraint)
Singularities	None	Gimbal lock	None
Composition	Matrix multiply	Complex formula	Quaternion multiply
Interpolation	Not directly possible	Direction ambiguity	SLERP (smooth, unique)
Numerical drift	Needs re-orthogonalization	N/A	Just re-normalize
Storage	9 floats	3 floats	4 floats

4.2 Advantages of Quaternions

1. **No gimbal lock** — singularity-free for all orientations
2. **Smooth interpolation** — SLERP gives constant angular velocity
3. **Compact** — only 4 numbers (vs 9 for a rotation matrix)
4. **Numerically stable** — re-normalizing a quaternion is cheap ($\mathbf{q}/\|\mathbf{q}\|$), while re-orthogonalizing a matrix is expensive
5. **Composition is fast** — 16 multiplications vs 27 for matrix multiply

4.3 Where You Will See Them

- **ROS**: `geometry_msgs/Quaternion` is the standard orientation message
- **Python**: `spatialmath.UnitQuaternion`
- **FANUC**: Internal orientation storage
- **Game engines**: Unity, Unreal Engine
- **Aerospace**: Attitude control, inertial navigation

5 SLERP: Spherical Linear Interpolation

5.1 The Problem

Given two orientations $\mathbf{q}_0$ and $\mathbf{q}_f$, find a smooth path between them.

5.2 The Formula

$$\text{SLERP}(\mathbf{q}_0, \mathbf{q}_f, s) = \frac{\sin\left((1-s)\,\Omega\right)}{\sin\Omega}\,\mathbf{q}_0 + \frac{\sin\left(s\,\Omega\right)}{\sin\Omega}\,\mathbf{q}_f$$

where:

- $s \in [0, 1]$ is the interpolation parameter (progress)
- $\Omega = \cos^{-1}(\mathbf{q}_0 \cdot \mathbf{q}_f)$ is the angle between the two quaternions

At $s = 0$ we get $\mathbf{q}_0$, at $s = 1$ we get $\mathbf{q}_f$.

5.3 Properties

1. **Constant angular velocity** — the orientation changes at a uniform rate
2. **Shortest path** — always takes the geodesic on the rotation hypersphere
3. **Unit quaternion output** — the result is always a valid rotation
4. **No gimbal lock** — works for any pair of orientations

5.4 Practical Note: Check the Dot Product

Before SLERP, compute $\mathbf{q}_0 \cdot \mathbf{q}_f$. If it is **negative**, negate one quaternion:

$$\text{if } \mathbf{q}_0 \cdot \mathbf{q}_f < 0, \text{ replace } \mathbf{q}_f \leftarrow -\mathbf{q}_f$$

This ensures SLERP takes the **short way** around (since $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation).

6 Python Examples

6.1 Creating Quaternions

```python
from spatialmath import UnitQuaternion, SO3
import numpy as np

# From axis-angle
q1 = UnitQuaternion.AngVec(np.pi/2, [0, 0, 1])  # 90° about z
print(q1)  # 0.7071 << 0.0000, 0.0000, 0.7071 >>

# From rotation matrix
R = SO3.RPY([30, 45, 60], unit='deg')
q2 = UnitQuaternion(R)

# Identity
q_id = UnitQuaternion()
```

6.2 Operations

```python
# Composition (multiply)
q3 = q1 * q2

# Inverse
q_inv = q1.inv()  # same as conjugate for unit quaternions

# Rotate a vector
v = [1, 0, 0]
v_rotated = q1.R @ v  # convert to matrix first
# or: v_rotated = q1 * v  (spatialmath supports this)

# Convert to/from rotation matrix
R = q1.R          # quaternion → 3×3 matrix
q = UnitQuaternion(R)  # matrix → quaternion
```

6.3 SLERP

```python
# Interpolate between two orientations
q_start = UnitQuaternion.RPY([0, 0, 0], unit='deg')
q_end = UnitQuaternion.RPY([90, 45, -60], unit='deg')

# Single interpolation at s = 0.5 (halfway)
q_mid = q_start.interp(q_end, 0.5)

# Full trajectory (50 steps)
trajectory = [q_start.interp(q_end, s) for s in np.linspace(0, 1, 50)]

# Using roboticstoolbox for full pose interpolation
import roboticstoolbox as rtb
from spatialmath import SE3
```

```
T0 = SE3.Trans(0.4, 0, 0.3) * SE3.RPY([0, 0, 0], unit='deg')
Tf = SE3.Trans(0.4, 0.2, 0.3) * SE3.RPY([0, 0, 90], unit='deg')

# ctraj does linear position + SLERP orientation
traj = rtb.ctraj(T0, Tf, 50)
```

7 Summary

- A **unit quaternion** $\mathbf{q} = (\cos\frac{\theta}{2},\ \sin\frac{\theta}{2}\,\hat{\mathbf{k}})$ represents a rotation by θ about axis $\hat{\mathbf{k}}$
- Quaternion **multiplication** composes rotations (order matters)
- The **conjugate** $\mathbf{q}^*$ gives the inverse rotation
- **SLERP** interpolates orientations with constant angular velocity
- Always check $\mathbf{q}_0 \cdot \mathbf{q}_f$ sign before SLERP to take the short path
- In Python: `spatialmath.UnitQuaternion`